Software Engineering in Practice

Lab Assignment

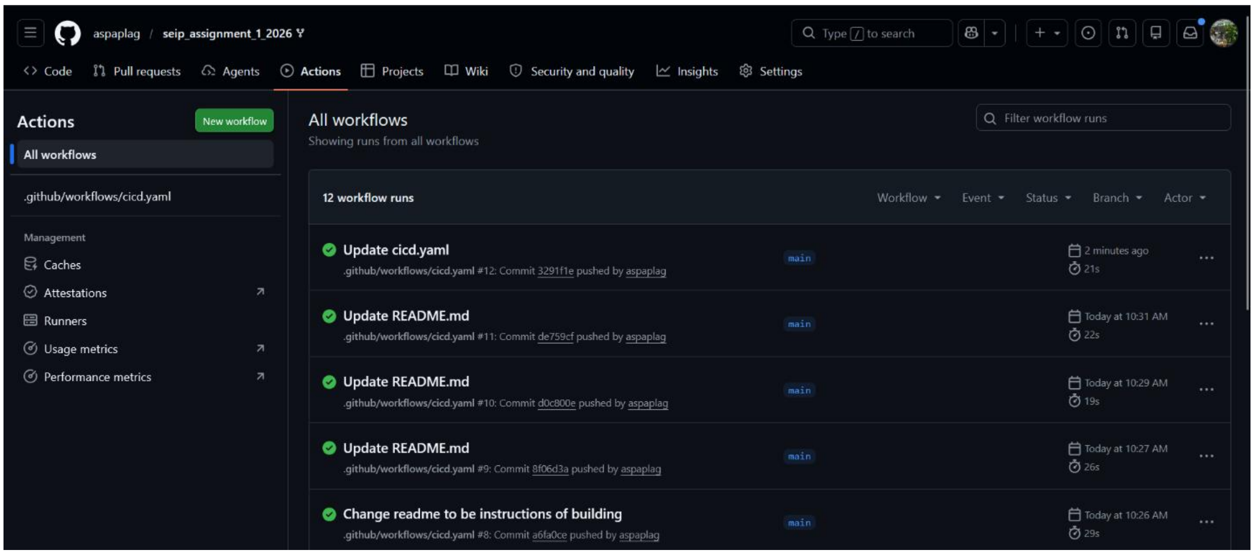
Professor: D. Spinellis

Aspasia Plangesi (8230122)

1. GitHub Repository Link

The code for this assignment is available at:
https://github.com/aspaplag/seip_assignment_1_2026

2. Successful CICD workflow run



3. Cluster State Proof

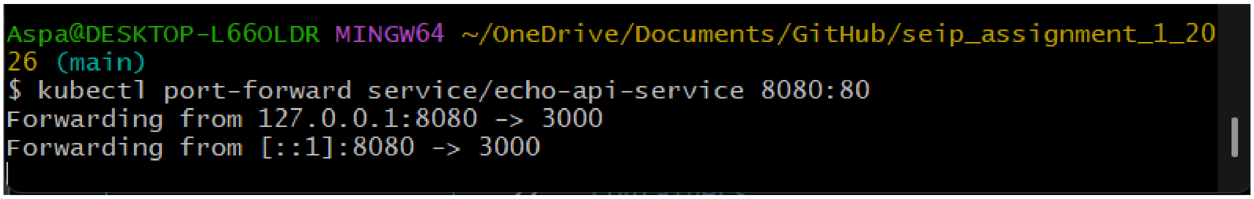
```
MINGW64:/c/Users/mprez/OneDrive/Documents/GitHub/seip_assignment_1_2...
26 (main)
$ kubectl get all -n default
NAME                             READY   STATUS    RESTARTS   AGE
pod/echo-api-7664dcbccb-k6gvc    1/1     Running   0          2m18s
pod/echo-api-7664dcbccb-twvg5    1/1     Running   0          2m6s
pod/echo-api-7664dcbccb-wtrg9    1/1     Running   0          2m9s

NAME                           TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)
 AGE
service/echo-api-service       ClusterIP   10.103.83.60   <none>        80/TCP
 49m
service/kubernetes             ClusterIP   10.96.0.1      <none>        443/TCP
 49m

NAME                           READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/echo-api       3/3     3            3           47m

NAME                                      DESIRED   CURRENT   READY   AGE
replicaset.apps/echo-api-5b84fcc9bc       0         0         0       5m16s
replicaset.apps/echo-api-6868cb6d5b       0         0         0       47m
replicaset.apps/echo-api-7664dcbccb       3         3         3       2m18s
replicaset.apps/echo-api-7b5947d788       0         0         0       2m37s
replicaset.apps/echo-api-f6bc68c46        0         0         0       4m52s
```

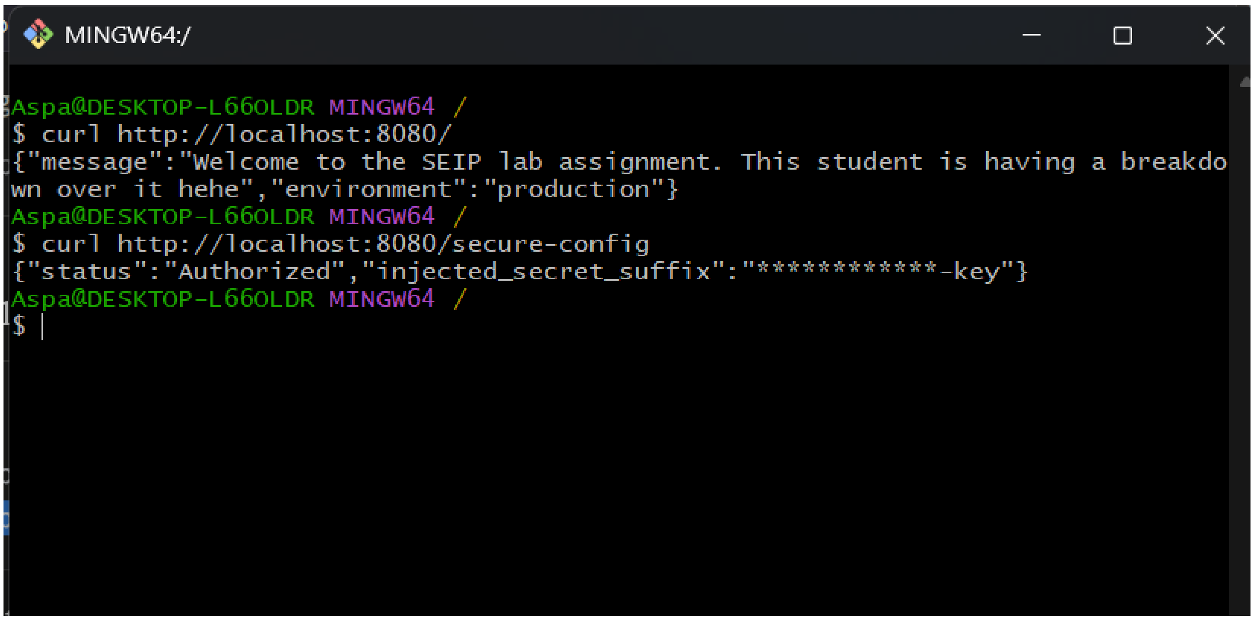
```
Aspa@DESKTOP-L66OLDR MINGW64 ~/OneDrive/Documents/GitHub/seip_assignment_1_20
26 (main)
$ kubectl get configmap,secret
NAME                          DATA   AGE
configmap/echo-api-config     2      49m
configmap/kube-root-ca.crt    1      50m

NAME                        TYPE     DATA   AGE
secret/echo-api-secret      Opaque   1      47m

Aspa@DESKTOP-L66OLDR MINGW64 ~/OneDrive/Documents/GitHub/seip_assignment_1_20
26 (main)
$ |
```

Proof of successful image pull:

```
for 300s
Events:
  Type    Reason     Age    From               Message
  ----    ------     ----   ----               -------
  Normal  Scheduled  2m11s  default-scheduler  Successfully assigned default/ech
o-api-6b4b486f86-k2ksm to minikube
  Normal  Pulling    2m10s  kubelet            Pulling image "ghcr.io/aspaplag/e
cho-api:latest"
  Normal  Pulled     2m8s   kubelet            Successfully pulled image "ghcr.i
o/aspaplag/echo-api:latest" in 2.03s (2.03s including waiting). Image size: 1345
90325 bytes.
  Normal  Created    2m8s   kubelet            Container created
  Normal  Started    2m8s   kubelet            Container started

Aspa@DESKTOP-L66OLDR MINGW64 ~/OneDrive/Documents/GitHub/seip_assignment_1_2026
(main)
$ |
```

4. Application Verification Proof





5. AI Usage & Future Engineering Report

5.1 AI integration

My engineering process went as such: I read the assignment, made brief google searches on Kubernetes, and the yaml file categories which I had not worked with before. I still did not have a completely clear understanding of tasks 2 and 3, so I used Claude to understand better. I prompted it to explain the requirements simply and walk me through the logic.

Afterwards, I wrote the dockerfile, workflow and yaml files on my own. I tried running them and the app failed due to errors in the manifests. I fed them into Claude and asked it to help me debug them and locate errors.

5.2 Utility Analysis

The LLM was most helpful in the beginning stages when trying to understand how Kubernetes manifest files work in order to create the desired state, how they interact and what their purpose is. It helped when viewing and trying to understand the error messages from the terminal by looking at the events. Finally, it was useful in understanding optimization, for example that "npm install" should be before "COPY . ." in the Dockerfile so it doesn't rerun, or that it would be better to have one job with separate steps instead of multiple jobs in the workflow.

5.3 Friction Points

There were a few instances where the LLM was not helpful.

At some point when executing the commands for the pods there was an issue with "ImagePullBackOff" because the container was failing to pull the image. The pipeline was still broken so the image had not been pushed, as I figured out later.

After the first two suggestions didn't work, it started spiraling, suggesting extremely complicated and unnecessary things such as using a PAT or finding workarounds to load the image locally instead of pulling as instructed.

During the cicd.yaml debugging, it also falsely marked (hallucinated) the newer action versions I had used as non-existent and bumped them down to older versions (had outdated information), and it also missed an indentation issue which broke the pipeline. I figured both of these through the failed workflow messages. One was a warning that the versions Claude had provided as the correct ones would stop being supported next month, so I reverted to the newer versions I had found on my own. The second was an error message pointing out wrong indentation in 2 lines, so I looked over the indents and fixed them manually.

Overall, although there were moments when it was very helpful in understanding the assignment and filling knowledge gaps, it often falls short in detail-oriented tasks.

5.4 Future Architectural Outlook

If this was a proper application with users, I would focus on automating a larger part of the process.

Firstly, I would include an Ingress controller to replace the manual port-forwarding, so as to handle traffic and routing to the service. Port forwarding only works while the command is running in my terminal and it only forwards to one service at a time. An Ingress controller, such as NGINX Ingress, acts as a single entry point into the cluster and routes incoming traffic to the correct service based on the hostname or URL path. This means users could reach the application through a normal web address and multiple services could be exposed through one entry point.

Moreover, I would focus on adding a GitOps workflow using a tool like ArgoCD. Currently I apply the manifests manually. This is not practical as it depends on a person running the command each time and there is no way to log the changes made over time. With GitOps, the cluster would automatically match its state to whatever is committed in the repository, which would help with the rollback as it would only be reverting to a previous commit in the repo.